# Diseño de Algoritmos – Práctica 0
# Medida del tiempo de ejecución

Francisco Palomo Lozano

Version 3.3

## Índice

**1. Introducción** **3**

**2. Tipos de medida** **3**

**3. Factores que influyen en la medida** **4**

**4. Formas de medir el tiempo** **4**
4.1. Medida directa . . . . . 5
4.2. Medida indirecta . . . . . 6
4.3. Medida adaptativa . . . . . 7

**5. Elección de los ejemplares de entrada** **8**

**6. Ejemplo: números de Fibonacci** **9**
6.1. Organización del código . . . . . 10
6.2. Makefile . . . . . 10
6.3. Programas . . . . . 14
6.4. Pruebas realizadas . . . . . 15
6.5. Gráficas . . . . . 16
6.6. Resultados obtenidos . . . . . 20

## Código

1. `cronometro.hpp` . . . . . 6
2. `makefile` . . . . . 13
3. `fibonacci-1.cpp` . . . . . 14
4. `fibonacci-2.cpp` . . . . . 14
5. `prueba-1.cpp` . . . . . 15

6. prueba-2.cpp . . . . . 16
7. graficas.plot . . . . . 18
8. graficas-pdf.plot . . . . . 19

# 1. Introducción

Para resolver un problema, pueden existir, en general, distintos algoritmos disponibles. La cuestión está en compararlos y elegir aquel que resulte más adecuado.

En la práctica, esta elección no es sencilla y suele estar condicionada por diversos factores, que perfilan el significado de qué es lo que se entiende por «más adecuado» en una determinada situación. Normalmente, los factores más apremiantes serán aquellos relacionados con el consumo de algún tipo de recurso: en la mayor parte de las ocasiones, memoria y, sobre todo, *tiempo de ejecución*.

Es en este último recurso en el que centraremos nuestra atención. El enfoque empírico para el análisis de algoritmos es, en esencia, muy simple: consiste en programar los distintos algoritmos y, como si de una competición se tratara, probar cada uno de ellos sobre una serie de ejemplares del problema a resolver para comparar sus tiempos de ejecución en idénticas condiciones. Para ello es necesario aprender a realizar correctamente la medida de dichos tiempos.

Tomaremos como ejemplo dos algoritmos sencillos para calcular números de Fibonacci y conduciremos una serie de experimentos a partir de los que será posible establecer conclusiones sobre la eficiencia temporal de ambos.

Posteriormente, justificaremos estos resultados a través de la teoría disponible. Esto es absolutamente necesario, ya que los resultados empíricos por sí mismos no bastan.

# 2. Tipos de medida

Una magnitud física, como, por ejemplo, el tiempo de ejecución de un programa, es el producto de un valor numérico por una unidad o magnitud de referencia.

El objeto de la *medida* es la determinación del valor numérico de la magnitud física en cuestión. Para ello, será necesario emplear un proceso de medida que permita obtener dicho valor. Pero hay que tener en cuenta que ningún proceso de medida es absolutamente exacto y que, por tanto, el valor real de una magnitud es imposible de determinar. Todo lo más que podemos esperar es obtener medidas lo suficientemente aproximadas para nuestros propósitos.

Una *medida directa* es aquella en la que el valor de la magnitud a medir se obtiene directamente. Por lo tanto, en una medida de este tipo se observa directamente el valor numérico en un instrumento de medida adecuado.

Una *medida indirecta* es aquella en la que el valor de la magnitud a medir se obtiene a partir de las medidas de otras magnitudes relacionadas con ella. En consecuencia, una medida de este tipo implica la manipulación matemática de una o varias medidas directas.

## 3. Factores que influyen en la medida

La diferencia entre el valor real y el valor medido es el *error* de la medida o, más precisamente, su *error absoluto*. El valor exacto del error es siempre desconocido, ya que el valor real de la magnitud medida también lo es, pero su valor absoluto puede acotarse: esta cota es la *incertidumbre* de la medida. Abusando del lenguaje, se suele emplear también el término «error» para indicar la incertidumbre.

Al error de una medida pueden contribuir diversas causas que, según su naturaleza, podemos clasificar en dos categorías: sistemáticas y accidentales.

Los *errores sistemáticos* tienen su origen en defectos del método o proceso de medida, o del instrumento utilizado, que dan lugar a una desviación de los resultados que se produce permanentemente y en un mismo sentido.

Los *errores accidentales* tienen su origen en causas ajenas a nuestro control que alteran los resultados a veces por defecto y otras por exceso. Habitualmente, ya que son impredecibles, se les supone aleatoriamente distribuidos y, en tal caso, se les denomina *errores aleatorios*.

La *resolución* de un instrumento de medida es el valor mínimo que ha de tener la magnitud a medir para poder ser detectada por el instrumento. Todo instrumento tiene una resolución por debajo de la cual el error cometido puede ser tan grande como el valor medido, haciendo que este sea completamente inútil.

También hay que tener en cuenta que el acto de medir influye siempre en la magnitud medida; esto introduce cierta incertidumbre en nuestros resultados. Cuanto mayores sean las magnitudes a medir, menor será en comparación la incertidumbre intrínseca a la realización de la medida.

## 4. Formas de medir el tiempo

Al analizar empíricamente un algoritmo, es necesario programarlo y realizar una serie de experimentos con distintos ejemplares de entrada, midiendo los tiempos de ejecución resultantes.

Para medir el tiempo de ejecución de un programa completo, puede emplearse la orden `time`, presente en los sistemas operativos tipo UNIX, que permite obtener, generalmente, información adicional sobre otros recursos consumidos por la ejecución del programa. Sin embargo, este método es muy poco flexible: en muchas ocasiones estamos interesados en medir el tiempo de un determinado fragmento de código que no deseamos aislar en un programa independiente. Además, suele ser poco preciso.

Cabe también la posibilidad de emplear un *perfilador*. No obstante, los perfiladores poseen una serie de limitaciones y sus datos pueden ser difíciles de interpretar y de postprocesar (para obtener gráficas, etc.). Un perfilador binario como Gprof posee induda-

bles ventajas, como permitirnos automatizar el proceso de instrumentación del código objeto, pero suele poseer una resolución relativamente pobre, puesto que su objetivo no es la medida precisa de los tiempos de ejecución; tampoco permite perfilar distintos hilos de ejecución y, por último, solo permite medir el *tiempo de usuario*.

Otra posibilidad, que es la que emplearemos, consiste en medir el tiempo de ejecución «por diferencia». Esto puede lograrse en C++ de dos formas, principalmente.

La primera es incluir la cabecera `<chrono>`, desde donde tenemos disponible la biblioteca estándar específica para la gestión del tiempo en el espacio de nombres `std::chrono`.

La segunda, más sencilla, heredada de C, es emplear la función `clock` de la biblioteca estándar, declarada en `<ctime>`, al igual que el tipo **`clock_t`** y la macro `CLOCKS_PER_SEC`. Esta función devuelve la mejor aproximación disponible del tiempo transcurrido desde un cierto instante de tiempo relacionado exclusivamente con la ejecución del programa. El resultado es un **`clock_t`**, un tipo aritmético entero que depende del sistema, cuyas unidades se denominan *clocks* o «pulsos». Para hallar el tiempo en segundos, basta dividir los pulsos entre el número de pulsos por segundo, disponible en `CLOCKS_PER_SEC`.

Como, normalmente, lo que deseamos calcular es el tiempo transcurrido entre dos puntos del programa, hay que obtener primero la diferencia entre el número de pulsos medidos en los instantes inicial y final. Para simplificar todo el proceso, podemos utilizar la clase `cronometro`, que hemos definido en `cronometro.hpp`. Esta clase nos permite crear un cronómetro que podemos activar y parar a voluntad. En cualquier momento podemos comprobar el tiempo transcurrido desde su última activación o, si lo hemos parado, el tiempo medido desde su última activación hasta el momento en que se paró.

## 4.1. Medida directa

Para realizar una *medida directa* del tiempo que tarda un experimento programado mediante un determinado fragmento de código, basta utilizar un objeto cronómetro de la siguiente forma:

```
cronometro c;
c.activar();
// ... fragmento a cronometrar ...
c.parar();
auto t = c.tiempo();
```

Las arquitecturas actuales están especialmente diseñadas para que la medida del tiempo sea precisa y se consiguen sin esfuerzo resoluciones del orden del microsegundo, es decir, una millonésima de segundo. Por ejemplo, en nuestro sistema, la resolución que arroja `clock` es, aproximadamente, esa.[1]

[1] En las primeras versiones de LINUX, la resolución de `clock` era muy escasa: la tarea de reloj programaba el temporizador para generar interrupciones con una frecuencia de 100 Hz, es decir, con un periodo de $1/100 = 0{,}01$ s. Hoy en día, los microprocesadores incorporan relojes de gran precisión.

```hpp
// Clase para cronometrar.

#ifndef CRONOMETRO_HPP_
#define CRONOMETRO_HPP_

#include <ctime>

using std::clock;
using std::clock_t;

class cronometro {
public:
  cronometro(): activo(false) {}
  void activar() { activo = true; t0 = clock(); }
  void parar() { if (activo) { t1 = clock(); activo = false; } }
  double tiempo() const { return ((activo ? clock() : t1) - t0) / pps; }
private:
  bool activo;     // Estado de actividad del cronómetro.
  clock_t t0, t1;  // Tiempos inicial y final.
  static constexpr double pps = CLOCKS_PER_SEC;
};

#endif
```

Fichero 1: `cronometro.hpp`

No obstante, este método de medida no es adecuado si el tiempo a medir no es lo suficientemente grande comparado con la resolución del cronómetro. Hay que evitar realizar medidas directas en tales situaciones.

## 4.2. Medida indirecta

Para solventar este problema, se puede utilizar la siguiente técnica de *medida indirecta*: medir el tiempo total de un cierto número de repeticiones del experimento y dividirlo entre el número de repeticiones realizadas para obtener el tiempo del experimento. De este modo, si el número de repeticiones es lo suficientemente alto, el tiempo medido directamente será notablemente superior a la resolución del instrumento de medida y se disminuirá el error.

Por ejemplo, suponiendo que la resolución fuera 1 ms, si quisiéramos obtener el tiempo de un experimento con una precisión de 1 µs tendríamos que medir el tiempo total de 1000 repeticiones de ese mismo experimento y dividirlo entre 1000. Lo mismo ocurriría si la resolución fuera de 1 µs y deseáramos conseguir una precisión de 1 ns.

```
const int r = 1000;
cronometro c;
c.activar();
for (int i = 0; i < r; ++i) {
  // ... fragmento a cronometrar ...
}
c.parar();
auto t = c.tiempo() / r;
```

Es muy importante que todas las repeticiones sean idénticas, es decir, que correspondan al mismo experimento, de lo contrario, el tiempo final resultante, puede no guardar relación con el tiempo que deseamos medir. En el caso de *algoritmos deterministas*, esto es sencillo, pues basta emplear exactamente los mismos datos de entrada en cada repetición y repetir el experimento en idénticas condiciones.[2]

No obstante, en el *análisis de algoritmos en el caso promedio* puede emplearse también una medida indirecta, en cuyo caso deberá disponerse de una muestra significativa de ejemplares para cada tamaño de entrada a analizar. El tiempo para cada ejemplar de un mismo tamaño puede ser distinto, pero esto no supone un inconveniente, pues estamos interesados en su tiempo promedio, no en sus tiempos individuales. La ejecución de cada ejemplar constituye un experimento que ha de realizarse en idénticas condiciones.

La situación es análoga en el caso de *algoritmos indeterministas*, por ejemplo, cuando la ejecución de un algoritmo depende internamente del azar.

Sin embargo, repetir un experimento un número de veces elevado puede hacer que el tiempo total de experimentación sea muy alto. El problema radica en que no se suele tener, a priori, una idea del tiempo que va a tardar el experimento y, en consecuencia, tampoco se conoce el número de repeticiones adecuado.

## 4.3. Medida adaptativa

Para medir el tiempo de un algoritmo determinista con precisión, se puede emplear un *esquema adaptativo de medida*: en vez de repetir cada experimento un número fijo de veces, se repite durante el tiempo necesario para garantizar la realización de un número suficiente de repeticiones y se promedia el tiempo total entre dicho número.

Por ejemplo, si el máximo error absoluto que se comete es $0{,}000001$ (supongamos que el único error cometido se debe a la resolución del cronómetro) y se desea garantizar un error relativo máximo del $1‰$ el tiempo total mínimo de experimentación sería $0{,}000001/0{,}001 + 0{,}000001$.

```
cronometro c;
long int r = 0;
```

[2] Debe prestarse especial atención si el experimento se ejecuta en un ordenador portátil, pues su rendimiento puede depender del perfil de ahorro de energía.

```
const double e_abs = 1E-6,  // Máximo error absoluto cometido.
             e_rel = 0.001; // Máximo error relativo aceptado.
c.activar();
do {
  // ... fragmento a cronometrar ...
  ++r;
} while (c.tiempo() < e_abs / e_rel + e_abs);
c.parar();
auto t = c.tiempo() / r;
```

La realización de repeticiones tiene otro efecto beneficioso, ya que ayuda también a disminuir cierto tipo de errores accidentales, como los debidos a la aparición puntual de un pico en la carga del sistema. Téngase en cuenta que en un sistema multitarea, una situación en la que la carga del sistema sea excesiva puede hacer variar el tiempo de ejecución de los procesos.

El esquema adaptativo debe emplearse cuando al analizar el tiempo para distintos ejemplares de entrada, los ejemplares de mayor tamaño tengan tiempos apreciables de por sí y sea un desperdicio repetir su ejecución, mientras que los de menor tamaño tengan tiempos despreciables y requieran muchas repeticiones. En tal caso, si se desea una medida precisa del tiempo de todos los ejemplares, incluidos los de menor tamaño, un esquema adaptativo es lo más apropiado.

## 5. Elección de los ejemplares de entrada

El tiempo que tarda un algoritmo en ejecutarse puede depender de la entrada concreta que reciba y no solo del tamaño de esta. El hecho de que muchos algoritmos se analicen respecto al tamaño de la entrada responde a una simplificación práctica necesaria, pero no hay que olvidar que para ejemplares de entrada del mismo tamaño los tiempos pueden ser muy distintos.

Es por ello, que se distinguen diversos casos de análisis: *mejor caso*, *peor caso* y *caso promedio*. Estos permiten clasificar las posibles entradas en familias que poseen tiempos de ejecución similares para cada tamaño.

Por lo tanto, antes de analizar empíricamente un algoritmo en función del tamaño de su entrada, hay que tener claro qué caso se desea analizar y cuáles son los ejemplares de entrada que lo producen; entonces podrán realizarse los experimentos a partir de una muestra de dichos ejemplares.

Al realizar un *análisis en el caso promedio*, los ejemplares de entrada se seleccionan *aleatoriamente* para cada tamaño de la entrada, por lo que hay que considerar que su *distribución de probabilidad* puede influir en los resultados. En ausencia de otros requisitos, se emplea una *distribución uniforme*, en la que, para un tamaño dado, cada posible ejemplar tiene la misma probabilidad de ser seleccionado que el resto.

Es necesario insistir en que, si se realiza una *medida indirecta* del tiempo de ejecución de un algoritmo sobre un ejemplar de entrada, ha de tenerse siempre la precaución de *realizar cada repetición sobre ese mismo ejemplar*. Esto implica que, por ejemplo, si el algoritmo modifica su entrada, puede ser necesario almacenar esta para poder realizar la siguiente repetición en igualdad de condiciones.

Igualmente, cuando se van a comparar dos algoritmos distintos que resuelven un mismo problema, hay que ejecutarlos exactamente sobre los mismos ejemplares de entrada, de forma que puedan compararse ejecución a ejecución; no basta hacerlo sobre ejemplares del mismo tamaño.

Las indicaciones anteriores son muy importantes. Solo siguiéndolas resultará posible comparar los resultados experimentales obtenidos en el laboratorio con los teóricos.

## 6. Ejemplo: números de Fibonacci

Para ilustrar lo explicado en secciones anteriores, nos disponemos a analizar empíricamente la eficiencia temporal de dos algoritmos para calcular números de Fibonacci.

Escribiremos una serie de programas y emplearemos diversas herramientas que nos ayudarán a medir los tiempos de los algoritmos para distintos valores de la entrada y a trazar gráficas que nos faciliten su comparación. Finalmente, explicaremos los datos experimentales obtenidos en función de los resultados teóricos correspondientes.

Recuérdese la definición de $F_n$, el $n$-ésimo número de Fibonacci:

$$F_n = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F_{n-1} + F_{n-2}, & n > 1. \end{cases}$$

El primer algoritmo se obtiene trivialmente a partir de esta definición recursiva. El segundo, algo más elaborado, resulta de aplicar *programación dinámica* a dicha definición, una técnica que estudiaremos en la próxima unidad.

```
f(n) → r
si n ≤ 1
   r ← n
si no
   r ← f(n − 1) + f(n − 2)
```

```
f(n) → r
(a, b) ← (0, 1)
mientras n > 1
            c ← a + b
            (a, b) ← (b, c)
            n ← n − 1
si n = 0
   r ← a
si no
   r ← b
```

## 6.1. Organización del código

El trabajo se organiza en una serie de ficheros que se distribuyen junto a la práctica:

`makefile` *Makefile* que incorpora, entre otros, los siguientes objetivos:

`all` — Es el objetivo por omisión, es decir, el que se activa cuando se ejecuta `make` sin más. Se encarga de hacer todo lo necesario para obtener los ficheros de datos de las gráficas.

`graficas` — Muestra las gráficas.

`graficas-pdf` — Crea los ficheros PDF correspondientes a las gráficas.

`clean` — Limpia el directorio de trabajo eliminando todo lo que puede volver a ser generado, salvo los ficheros de datos.

`clean-all` — Limpia completamente el directorio de trabajo.

`graficas.plot` Guion para GNU Plot que se emplea para mostrar las gráficas.

`graficas-pdf.plot` Guion para GNU Plot que se emplea para crear los ficheros PDF.

`natural.hpp` Contiene el tipo empleado para representar números naturales.

`fibonacci-1.cpp` Contiene la función correspondiente al primer algoritmo.

`prueba-1.cpp` Programa que realiza los experimentos correspondientes al primer algoritmo y escribe los resultados en la salida estándar.

`fibonacci-2.cpp` Contiene la función correspondiente al segundo algoritmo.

`prueba-2.cpp` Programa que realiza los experimentos correspondientes al segundo algoritmo y escribe los resultados en la salida estándar.

En las siguientes secciones se discutirá el contenido de cada uno de estos ficheros en detalle.

## 6.2. Makefile

La utilidad GNU Make está especialmente diseñada para ayudar a rehacer trabajos monótonos que deban repetirse cada vez que ciertos ficheros sean modificados. Esta es, precisamente, la situación que nos encontraremos en muchas ocasiones.

Para ello, se describen en un fichero una serie de *reglas*. Cada regla consta de *objetivos*, *dependencias* y *órdenes*. Solo los objetivos son obligatorios en una regla, y se separan de las dependencias por el carácter «`:`». Cada línea que describe las dependencias de uno o varios objetivos puede ir seguida de una serie de órdenes sangradas obligatoriamente con un tabulador.

El fichero `makefile` contiene todo lo necesario para compilar el código, ejecutarlo, medir los tiempos y representarlos gráficamente.

Cuando se requiera la realización de un objetivo, basta ejecutar `make`, que comprobará si alguna de sus dependencias ha sido modificada con posterioridad al objetivo (esto es, si el objetivo es más antiguo). En tal caso, el objetivo deberá rehacerse y sus órdenes ejecutarse, no sin antes comprobar y rehacer las dependencias del mismo que sean necesarias.

No todos los objetivos son necesariamente ficheros: existen «objetivos falsos», que no están asociados a ningún fichero y se emplean para desencadenar una serie de acciones (por ejemplo, la limpieza del directorio de trabajo). Los objetivos falsos `all`, `clean` y `clean-all` son tan comunes en los *makefiles* que no se suelen traducir sus nombres.

Al principio, se definen una serie de variables a cuyos valores se accede mediante `$`. Algunas de ellas, como `CXX` y `CXXFLAGS`, son especiales: `make` las utiliza en sus «reglas implícitas». Por ejemplo, una de las reglas implícitas de `make` dice que un módulo objeto se obtiene a partir de su fuente `.cpp` compilando con `$(CXX)` y con las opciones `$(CXXFLAGS)`. Por lo tanto, `make` conoce esta información y no hace falta suministrarla.

Existen otras variables especiales. Así, `$@` es el objetivo que se está tratando, `$^`, todas sus dependencias, y `$<` su primera dependencia. Los valores de las variables se pueden cambiar desde fuera. Por ejemplo, para compilar para depuración se haría:

```
% make CXXFLAGS=-g
```

Respecto a las opciones de mejora de código, en GNU C++, `-O0` indica «sin optimización», y `-O3`, «optimización plena». Por ejemplo, si ya se ha compilado el código para llevar a cabo un experimento sin optimización y, posteriormente, se desea recompilarlo para volver a realizar el experimento con optimización plena, puede hacerse:

```
% make clean
% make CXXFLAGS=-O3
```

```
# Compilador de C++ y opciones de compilación.

CXX = g++
override CXXFLAGS += -std=c++17 -Wall -Wextra

# Módulos objeto y ejecutables.

OBJS = prueba-1.o prueba-2.o fibonacci-1.o fibonacci-2.o
EXES = prueba-1 prueba-2

# Ficheros de tiempo y de gráficas.

TIEMPOS = prueba-1.tmp prueba-2.tmp
GRAFICAS = prueba-1.pdf prueba-2.pdf pruebas-1-y-2.pdf

# Por omisión, obtiene los ficheros de tiempo.

all: $(TIEMPOS)

# Obtención de los ejecutables.

prueba-1: prueba-1.o fibonacci-1.o
	$(CXX) -o $@ $(LDFLAGS) $^ $(LDLIBS)

prueba-2: prueba-2.o fibonacci-2.o
	$(CXX) -o $@ $(LDFLAGS) $^ $(LDLIBS)

# Obtención de los objetos.

$(OBJS): natural.hpp

prueba-1.o prueba-2.o: cronometro.hpp

# Obtención de los ficheros de tiempo.

prueba-1.tmp: prueba-1
	./$< | tee $@

prueba-2.tmp: prueba-2
	./$< | tee $@

# Obtención de las gráficas.

```

```
44 graficas:
45         gnuplot graficas.plot
46
47 graficas-pdf:
48         gnuplot graficas-pdf.plot
49
50 # Limpieza del directorio.
51
52 clean:
53         $(RM) $(EXES) $(OBJS) *~
54
55 clean-all: clean
56         $(RM) $(TIEMPOS) $(GRAFICAS)
```

Fichero 2: makefile

```cpp
#include "natural.hpp"

// Cálcula el n-ésimo número de Fibonacci por el algoritmo recursivo trivial.

natural f(natural n)
{
  return n <= 1 ? n : f(n - 1) + f(n - 2);
}
```

Fichero 3: `fibonacci-1.cpp`

```cpp
#include "natural.hpp"

// Cálcula el n-ésimo número de Fibonacci por programación dinámica.

natural f(natural n)
{
  natural a = 0, b = 1;
  while (n > 1) {
    const natural c = a + b;
    a = b;
    b = c;
    --n;
  }
  return n ? b : a;
}
```

Fichero 4: `fibonacci-2.cpp`

## 6.3. Programas

La programación de los algoritmos es directa y no merece mayor explicación, salvo por la elección del tipo `natural`. El código se incluye en `fibonacci-1.cpp` y `fibonacci-2.cpp`. En primera instancia, consideraremos que los números naturales tienen precisión limitada. Así, una posibilidad es que `natural.hpp` contenga simplemente:

```cpp
typedef unsigned long int natural;
```

Emplear precisión limitada es ciertamente una limitación, y valga la redundancia, puesto que no nos permite calcular exactamente más que unos cuantos números de Fibonacci (recuérdese que el número de dígitos de $F_n$ crece linealmente con $n$).

Visto de otro modo: ya que cada número se almacena en un **`unsigned long int`**, esto equivale a realizar los cálculos modularmente; no se calcula $F_n$, sino $F_n \bmod p$, siendo $p$ el siguiente número natural al máximo valor representable con este tipo.

```cpp
#include <iostream>
#include "natural.hpp"
#include "cronometro.hpp"

natural f(natural n);

// Genera la tabla de tiempos frente a n.

int main()
{
  cronometro c;
  for (natural n = 0; n <= 50; ++n) {
    c.activar();
    f(n);
    c.parar();
    std::cout << n << '\t' << c.tiempo() << std::endl;
  }
}
```

Fichero 5: prueba-1.cpp

Si no es esto lo que queremos y se desean números naturales de precisión ilimitada, es necesario programar una clase para su manejo o emplear una biblioteca especializada, ya que C++ no la posee primitivamente.

## 6.4. Pruebas realizadas

Analizaremos el tiempo de ejecución en función de $n$, el valor de la entrada. Decidimos variar $n$ desde $0$ hasta $50$ para cada algoritmo. Como se podrá comprobar, el tiempo total del experimento es apreciable.

En el caso del primer algoritmo podemos emplear una medida directa, aunque esta sea completamente imprecisa para los primeros valores de $n$, ya que rápidamente el tiempo de cálculo comienza a ser apreciable. El código necesario se incluye en prueba-1.cpp.

En el caso del segundo algoritmo los tiempos de cálculo son tan pequeños que la medida no debe realizarse directamente. Esto se debe a la escasa resolución de nuestro instrumento de medida. Podemos emplear la técnica de medida indirecta explicada repitiendo cada experimento el número de veces necesario para obtener la precisión deseada. El código necesario se incluye en el fichero prueba-2.cpp.

```cpp
#include <iostream>
#include "natural.hpp"
#include "cronometro.hpp"

natural f(natural n);

// Genera la tabla de tiempos frente a n.

int main()
{
  cronometro c;
  for (natural n = 0; n <= 50; ++n) {
    const int r = 1000;
    c.activar();
    for (int i = 0; i < r; ++i)
      f(n);
    c.parar();
    std::cout << n << '\t' << c.tiempo() / r << std::endl;
  }
}
```

Fichero 6: `prueba-2.cpp`

## 6.5. Gráficas

Emplearemos la herramienta GNU Plot para representar gráficamente los resultados obtenidos del análisis empírico de los algoritmos. Esta herramienta es muy completa, y merece la pena estudiarla por sí misma; al ejecutar `gnuplot` se dispone de ayuda integrada a través de la orden `help`.

El fichero `graficas.plot` recoge una sesión de GNU Plot cuyo objetivo es mostrar las gráficas correspondientes a los datos almacenados en los ficheros `.tmp`. Las órdenes que contiene pueden ser introducidas desde dentro del programa; no obstante, resulta muy cómodo y útil agruparlas en un *guion* y pasar este como parámetro desde el exterior al ejecutar `gnuplot`. Esto es lo que se hace exactamente dentro del *makefile*.

El formato de los ficheros de datos es muy sencillo. Son ficheros de texto donde cada línea contiene la abscisa y la ordenada de un punto, como en el siguiente fragmento de ejemplo:

```
⋮
48        32.5283
49        53.0689
50        86.8016
```

La opción `terminal` controla el dispositivo gráfico a través del que se generarán las gráficas. El dispositivo especial `qt` representa los datos en una ventana gráfica a través de la biblioteca Qt. Como se observa, es posible crear varias ventanas gráficas numerándolas. También se puede representar más de una gráfica simultáneamente en cada ventana, lo que resulta muy útil a efectos de comparación.

Dado que puede ser interesante obtener gráficas en formatos apropiados para su inclusión en otros documentos (como este), o para su impresión, resulta útil preparar un guion específico.

El guion `graficas-pdf.plot` es muy parecido al anterior. Las líneas clave son las que permiten cambiar la codificación de los caracteres, el dispositivo a través del que se generan los gráficos y el destino de la propia salida a la que se envían estos.

Gracias a la opción `encoding` es posible obtener correctamente las representaciones gráficas correspondientes a los caracteres *multibyte*, como los codificados en UTF-8.

Obtendremos código PDF. Esto se consigue cambiando la opción `terminal` por una terminal virtual que genera el código a través de una biblioteca apropiada. Por último, con la opción `output` se redirige la salida estándar al fichero indicado. Por omisión, los datos se envían a la salida estándar (salvo en el caso de que la terminal corresponda a un dispositivo especial).

```
# Título de cada eje.

set xlabel 'n'
set ylabel 't(n) (tiempo en segundos)'

# Rejilla.

set grid linestyle -1

# Posición de la leyenda.

set key left top

# Estilo de presentación (puntos interpolados linealmente).

set style data linespoints
set linetype 1 pointtype 5 pointsize 2 linecolor rgb 'blue'
set linetype 2 pointtype 7 pointsize 2 linecolor rgb 'red'

# Representación gráfica.

set terminal qt size 1920,1080 linewidth 4 persist

set terminal qt 0 title 'Fibonacci recursivo trivial'
plot 'prueba-1.tmp' title 'Fibonacci recursivo trivial'

set terminal qt 1 title 'Fibonacci dinámico'
plot 'prueba-2.tmp' title 'Fibonacci dinámico' linetype 2

set terminal qt 2 title 'Comparación entre ambas versiones'
plot 'prueba-1.tmp' title 'Fibonacci recursivo trivial', \
     'prueba-2.tmp' title 'Fibonacci dinámico'
```

Fichero 7: `graficas.plot`

```
⋮
# Codificación y terminal.

set encoding locale
set terminal pdf size 16cm,10cm linewidth 2

# Creación de los ficheros PDF.

set output 'prueba-1.pdf'
plot 'prueba-1.tmp' title 'Fibonacci recursivo trivial'

set output 'prueba-2.pdf'
plot 'prueba-2.tmp' title 'Fibonacci dinámico' linetype 2

set output 'pruebas-1-y-2.pdf'
plot 'prueba-1.tmp' title 'Fibonacci recursivo trivial', \
     'prueba-2.tmp' title 'Fibonacci dinámico'
```

Fichero 8: `graficas-pdf.plot`

| | |
|---|---|
| Microprocesador | Intel Core i7-8565U, 1,8 GHz |
| Memoria principal | DRAM LPDDR3 16 GiB, 2133 MT/s |
| Sistema operativo | Ubuntu 22.04 LTS |
| Compilador | GNU C++ 11.4.0 |

Cuadro 1: Equipo de experimentación.

### 6.6. Resultados obtenidos

Los experimentos se han realizado con el equipo consignado en el cuadro 1. Por supuesto, al repetir estos experimentos en el laboratorio o en su casa, presumiblemente con un equipo distinto y bajo condiciones también distintas, obtendrá datos diferentes.

Los resultados obtenidos para cada experimento aparecen en las gráficas que se presentan en la figura 1. Basta observar 1c para comprobar cómo el tiempo empleado por el segundo programa es cada vez más despreciable frente al correspondiente al primero.

Se observa un comportamiento exponencial de la función representada en 1a. Esto coincide plenamente con los resultados teóricos, ya que no es difícil comprobar que se realizan $\Theta(\phi^n)$ sumas y que ninguna otra operación se ejecuta asintóticamente más que la suma. Cuando cada suma se ejecuta empleando aritmética de precisión limitada (y así lo hace el programa), es una operación elemental y el tiempo real del programa asociado coincide, en orden, con el abstracto del algoritmo.

En efecto, sea $t(n)$ el número de sumas que realiza el primer algoritmo, su ecuación de recurrencia asociada es:

$$t(n) = \begin{cases} 0, & n \leq 1 \\ t(n-1) + t(n-2) + 1, & n > 1. \end{cases}$$

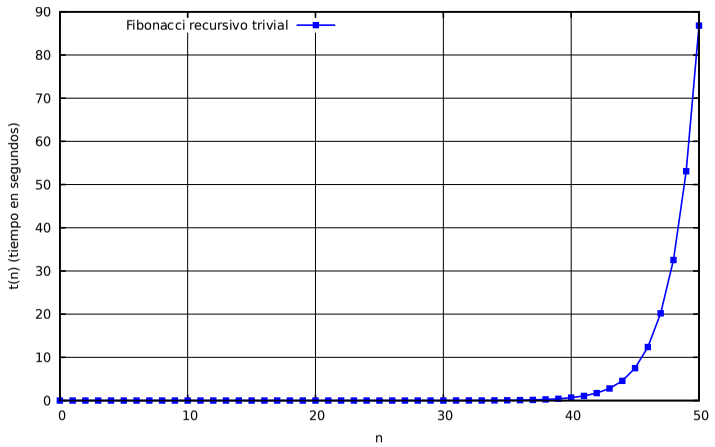
La solución viene descrita por

$$t(n) = \frac{1}{\phi + \phi^{-1}} \cdot (\phi^{n+1} - \phi^{-(n+1)}) - 1,$$

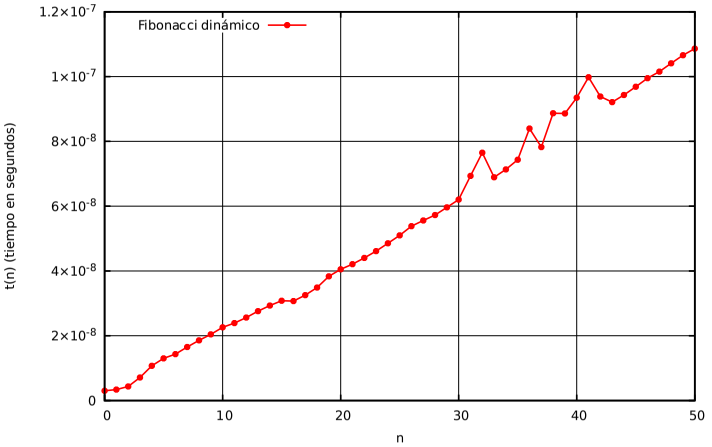
siendo $\phi$ el *número áureo*. Por lo tanto, $t(n) \in \Theta(\phi^n)$.

Por el contrario, la función representada en 1b tiene un comportamiento lineal. Esto se debe a que $t(n) = n - 1$ sumas (para $n \geq 1$) con el segundo algoritmo. Por idénticas razones a las ya comentadas, el tiempo real del programa asociado pertenecerá a $\Theta(n)$.

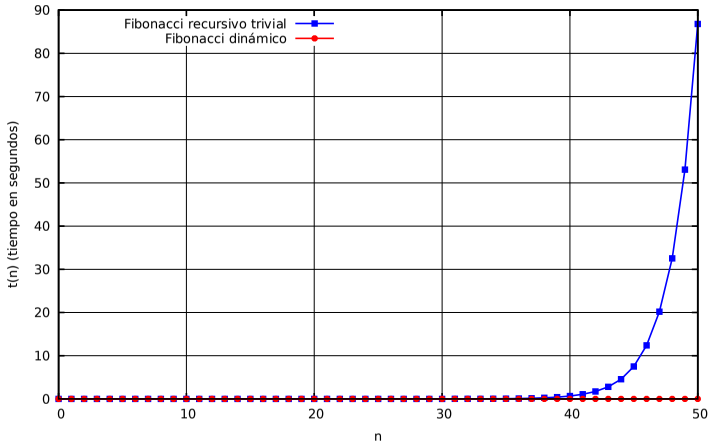
Figura 1: Resultados de los experimentos.




(a) Tiempos obtenidos con el primer algoritmo.




(b) Tiempos obtenidos con el segundo algoritmo.




(c) Tiempos obtenidos con ambos algoritmos.

## Ejercicios

1. Como se ha explicado, los programas presentados calculan los números de Fibonacci con precisión limitada. ¿Cuál es el mayor número de Fibonacci que puede calcular exactamente en su sistema? Compruebe qué valor se obtiene en lugar del número de Fibonacci que lo sigue y explique exactamente por qué se produce ese valor y no otro.

2. Los tiempos en segundos obtenidos en un cierto equipo para los siguientes valores de $n$ y cada algoritmo aparecen en la siguiente tabla:

| $n$ | $t_1$ | $t_2$ |
|---|---|---|
| 40 | 0,66 | $9{,}34 \cdot 10^{-8}$ |
| 41 | 1,07 | $9{,}98 \cdot 10^{-8}$ |
| 42 | 1,73 | $9{,}38 \cdot 10^{-8}$ |
| 43 | 2,79 | $9{,}21 \cdot 10^{-8}$ |
| 44 | 4,55 | $9{,}43 \cdot 10^{-8}$ |
| 45 | 7,51 | $9{,}68 \cdot 10^{-8}$ |
| 46 | 12,38 | $9{,}95 \cdot 10^{-8}$ |
| 47 | 20,19 | $1{,}01 \cdot 10^{-7}$ |
| 48 | 32,52 | $1{,}04 \cdot 10^{-7}$ |
| 49 | 53,06 | $1{,}06 \cdot 10^{-7}$ |
| 50 | 86,80 | $1{,}08 \cdot 10^{-7}$ |

   Repita los experimentos en el laboratorio. Debe observar que los nuevos tiempos obtenidos se diferencian de los anteriores, aproximadamente, en una constante multiplicativa. ¿Por qué? Calcule dicha constante y el porcentaje de mejora o empeoramiento que se produce en los tiempos de cada algoritmo.

3. Repita los experimentos con las distintas opciones de mejora de código del compilador (`-O1`, `-O2` y `-O3`) y compruebe su impacto sobre los tiempos de ejecución de cada algoritmo.

4. Repita los experimentos empleando un esquema adaptativo de medida y compruebe gráficamente que obtiene resultados similares.